数据结构课程设计报告

-------电话号码查找系统

软件学院软件工程专业 2013 级 3 班

姓名：许崇杨 学号：2013003xxxxx

任课教师：孔兰菊　　实验教师：胡洁萍 张晓梅 助教：

目 录

一、系统开发平台 2
1.1 题目： 2
1.2 开发工具 2
二、系统规划 2
2.1 任务陈述 2
2.2 任务目标 2
三、系统定义 3
四、需求分析 3
4.1 用户需求说明 3
4.2 系统需求说明 5
五、数据结构设计 5
5.1 逻辑结构及操作设计 5
5.2 存储结构 8
六、算法设计 10
6.1 链式哈希表 10
6.2 线性开放寻址 12
6.3 界面设计 14
七、测试和运行 14
7.1 测试一 14
7.2 测试二 15
八、总结 15
附. 参考文献 15
附. 程序清单 15

一、系统开发平台

1.1 题目：

设计散列表实现电话号码查找系统。
基本要求：
（1）设每个记录有下列数据项：电话号码、用户名、地址；
（2）从键盘输入各记录，分别以电话号码和用户名为关键字建立散列表；
（3）查找并显示给定电话号码的记录；
（4）查找并显示给定用户名的记录。
（5）尝试不同类型处理冲突的方法，考察平均查找长度的变化。

1.2 开发工具

使用 windows7 操作系统，采用 jdk 1.7(dk-7u75-windows-x64.exe) java 开发环境，IDE 采用 eclipse Luna，使用 java 高级程序设计语言开发。

二、系统规划

2.1 任务陈述

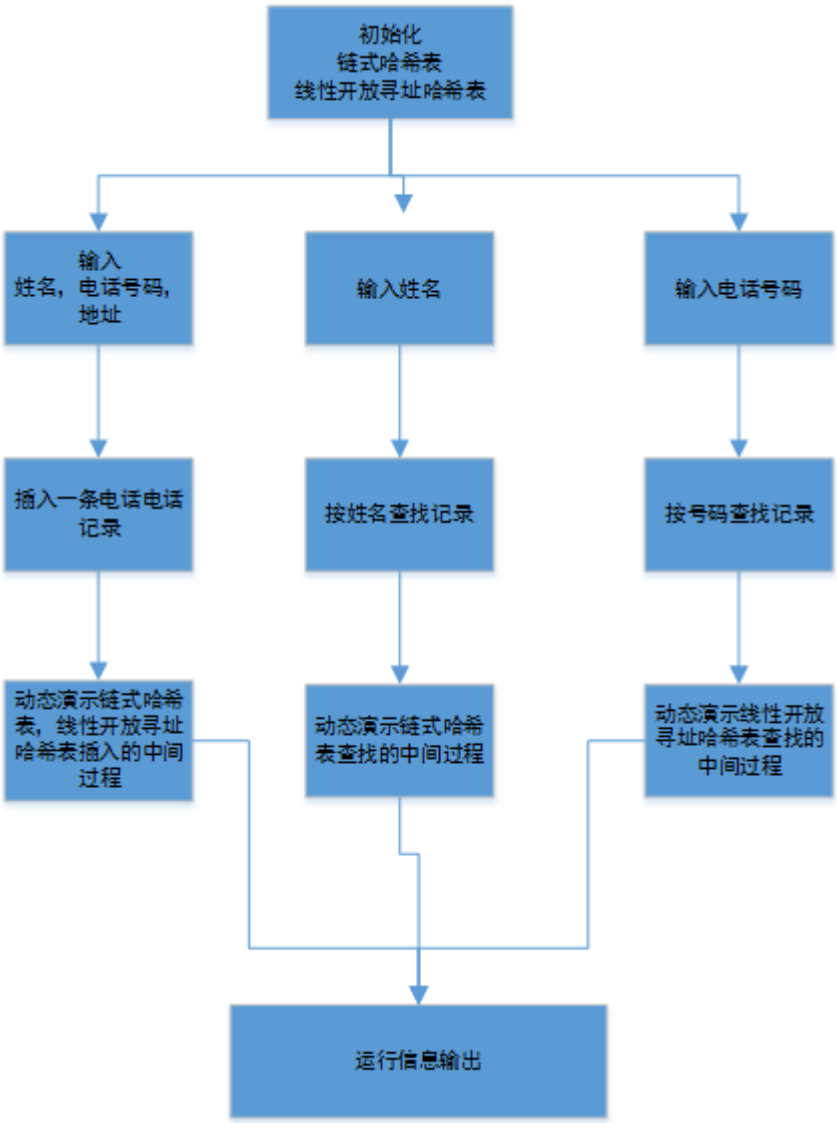
本系统用于哈希表中线性开放寻址和链式哈希两种解决冲突方式的动画演示，动画演示两种解决冲突方式各自的插入查找算法的中间过程。
系统通过插入一条电话记录（包含电话号码，姓名和地址），以及通过姓名或电话号码查找，演示线性开放寻址和链式哈希两种方式插入，查找算法的中间过程。
系统可以作为电话号码管理系统使用，也可以演示算法的中间过程，帮助理解算法的中间过程

2.2 任务目标

维护（插入，查找）线性开放寻址哈希表和链式哈希表
实现定位母桶的展示
实现插入过程的展示
实现查找过程的展示
实现算法执行过程信息的输出
实现单步执行的开关控制
实现哈希表规模的设置
实现姓名，电话号码，地址的文本框输入
实现插入，按姓名查找，按电话号码查找操作按钮

实现线性开放寻址哈希表和链式哈希表插入，查找算法的比较

三、系统定义



四、需求分析

4.1 用户需求说明

4.1.1 数据结构需求

电话记录，包含电话号码，姓名和地址
操作类型，插入，按姓名查找，按号码查找
哈希表的母桶个数
是否开启单步执行

插入算法需要 以姓名或号码为键，记录为值的键值对
查找算法需要姓名或号码
动画演示需要每个节点包含当前状态，以区分哈希表的不同状态进行绘制

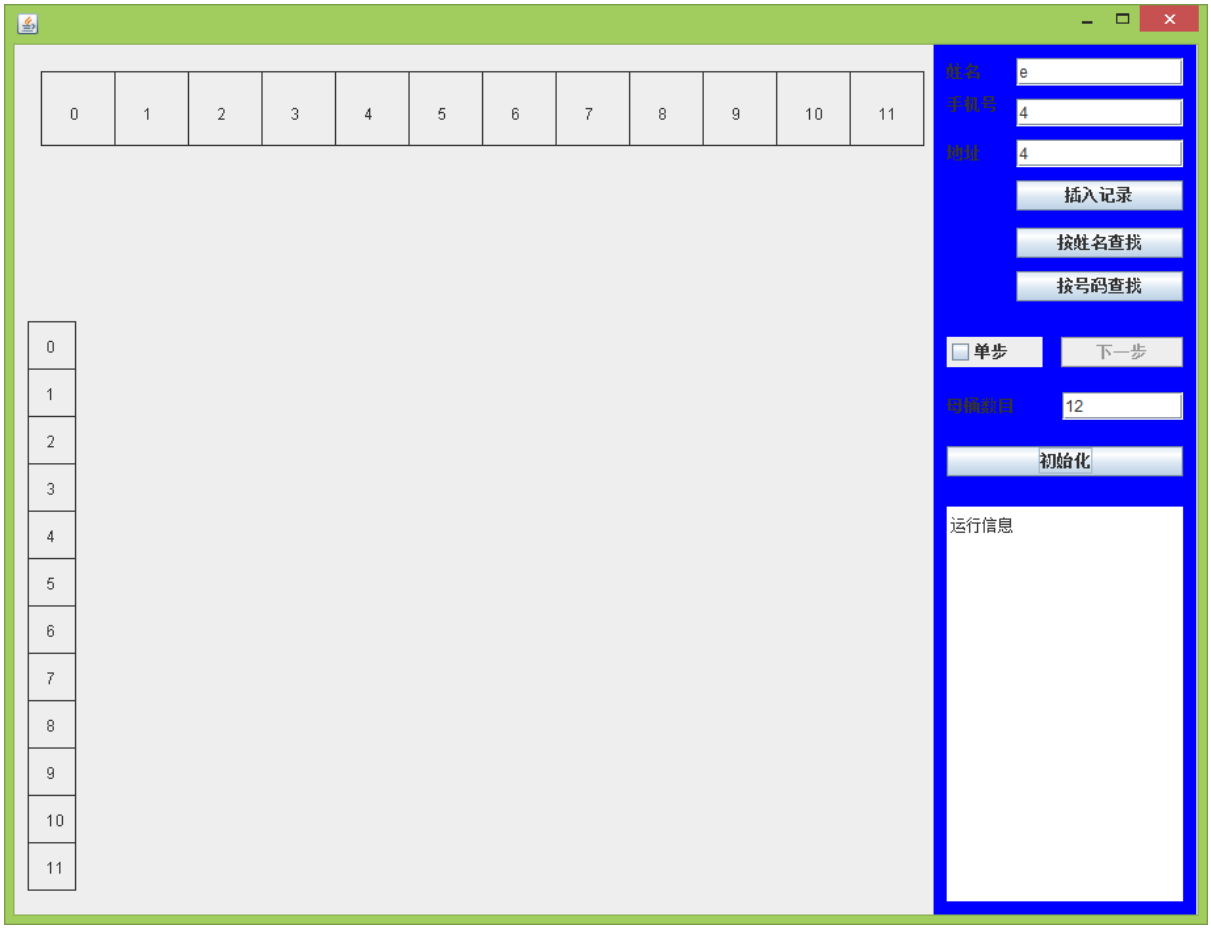
4.1.2 操作需求

数据录入
插入记录
输入母桶个数
选择单步执行
根据母桶个数初始化哈希表
按姓名查找
- 链式哈希
  - 定位母桶
  - 创建有序链表
  - 将节点插入有序链表
  - 展示引用位置的变化
  - 展示节点插入已确定位置的过程
- 线性开放寻址
  - 定位母桶
  - 展示探查可插入位置的过程
  - 展示插入

按号码查找
- 链式哈希
  - 定位母桶
  - 展示引用位置的变化
  - 展示键值与节点键值比较的过程
- 线性开放寻址
  - 定位母桶
  - 展示键值与节点键值的比较

4.1.3 界面需求

输入界面需求：3 个文本框输入姓名，电话号码，地址；插入按钮，按姓名查找按钮，按号码查找按钮，下一步按钮，初始化按钮
输出界面需求：两个面板分别用于展示链式哈希表和线性开放寻址哈希表操作的动态演示；文本面板用于输出运行信息



4.2 系统需求说明

关键事务插入查找操作以及动画演示
系统使用的高峰期是插入查找操作，低谷期是输入数据，选择单步执行，输入木桶的时期
解决问题的规模，不宜过大，木桶数目在 5-13 为宜

五、数据结构设计

5.1 逻辑结构及操作设计

5.1.1 电话记录

电话记录包含 姓名，电话号码，地址；操作包含初始化，获取姓名，电话号码，地址的操作。

对应抽象数据类型

AbstractDataType **Record**{

实例：包含姓名，号码，地址三个属性的对象

操作：

Create():创建一条记录

GetName():获取记录的姓名

GetNumber():获取记录的号码

GetAddress():获取记录的地址

}

5.1.2 键值对

键值对，关键字和值一一对应

以姓名为关键字或号码为关键字建立的键值对，关键字和记录一一对应，包含当前键值对所处的状态；操作包括初始化，获取关键字，获取值，设置值。对应抽象数据类型

AbstractDataType **Pair**{

实例：包含关键字和值以及状态的对象

操作：

Create():创建一个键值对

getKey():获取关键字

getValue():获取值

setValue(v):设置值

getState():获取当前 Pair 的状态

setState():设置当前 Pair 的状态

}

5.1.3 链表节点

链表节点，包含键值对，下一个链表节点的引用，当前节点的状态；操作包括初始化，获取状态，设置状态，获取键值对，设置键值对，获取下一个结点的引用，设置下一个节点的引用。对应抽象数据类型

AbstractDataType **PairNode**{

实例：包含一个 Pair 类型的对象，下一个 PairNode 节点的引用，当前的状态

操作：

Create();创建一个 PairNode

getState();获取状态

setState();设置状态

getPair();获取节点里的 Pair

setPair();这只节点里的 Pair

getNext();获取下一个

setNext();设置下一个

}

5.1.4 有序链表

有序链表，一组 PairNode 线性相连，按照键值的大小从小到大排列;操作包含初始化，插入，查找，绘画，获取链表大小

AbstractDataType **SortedChain{**

实例 :包含一组按关键字大小线性排列的 PairNode

操作：

Create();初始化

Insert(x);插入一个节点

Search(x);查找一个节点

Draw();绘制有序链表

Size();链表中元素的个数

}

5.1.5 链式哈希表

链式哈希表，将关键字按照某种映射关系映射到线性表的位置，线性表的每个位置有一个有序链表，发生冲突时，将冲突的元素保存在有序链表中。操作包括初始化，计算哈希值，插入，查找，绘制.

AbstractDataType **HashChain**{

实例：以关键字为索引，用链表解决冲突的元素集合

操作：

Create();//初始化

Hash(k);//计算哈希值

Insert(x);//插入 x

Search(k);//按关键字 k 查找

Draw();//绘制

}

5.1.6 线性开放寻址哈希表

线性开放寻址哈希表，将关键字按照某种映射关系映射到线性表的位置，发生冲突时，将冲突的元素沿着线性表向下找到没有元素的位置. 操作包括初始化，计算哈希值，插入，查找，绘制.

AbstractDataType **HashTable**{

实例：以关键字为索引，用线性开放寻址解决冲突的元素集合

操作：

Create();//初始化

Hash(k);//计算哈希值

Insert(x);//插入 x

Search(k);//按关键字查找

Draw();//绘制

}

5.2 存储结构

5.2.1Record 类型存储结构

```
public class Record {

    public String name;
    public long  phoneNum;
    public String address;
}
```

5.2.2Pair 类型存储结构

```
public class Pair<K,V> {
    public enum STATE {NEXT, STOP, NORMAL};
    K key;
    V value;
    STATE state = STATE.NORMAL;
    public STATE getState() ;
    public void setState(STATE state) ;
    public Pair(K key, V value);
    public K getKey();
    public V getValue();
    public void setValue(V value);
}
```

5.2.3PairNode 类型存储结构

```
public class PairNode<K,V> {
    public enum STATE {
        NOT_VISITING, VISITING, DESTINATION
    };
    public Pair<K,V> element=null;
    PairNode<K,V> next=null;
    private STATE state = STATE.NOT_VISITING;
    public STATE getState();
    public void setState(STATE state) ;
    public PairNode(Pair<K,V> pair) ;
    public PairNode(Pair<K,V> pair,PairNode<K,V> next);
    public PairNode(Pair<K,V>pair,PairNode<K,V>next,STATEstate);
}
```

5.2.4SortedChian 类型存储结构

```
public class SortedChain<K,V> {
    protected PairNode<K,V> first;
    protected int dSize;

    public SortedChain();
    public boolean empty();
```

```
    public int size() ;
    public void insert(Pair<K,V> pair, final MainFrame frame,
Thread thread);
    public Pair<K,V> search(K key, final MainFrame frame,Thread
thread);
    public void draw(MainFrame frame, Graphics g, int x0, int y0,
int cellSize);

}
```

5.2.5HashChain 类型存储结构

```
public class HashChain<K,V> {

    public SortedChain<K,V>[] table ;
    public int dSize;
    public int divisor;

    public HashChain(int theDivisor);
    public Pair<K,V> search(K key, MainFrame frame, Thread thread)
    public void insert(Pair<K,V> pair,MainFrame frame,Thread
thread)
    public SortedChain<K,V>[] getTable();
    private int hash(Object key);
}
```

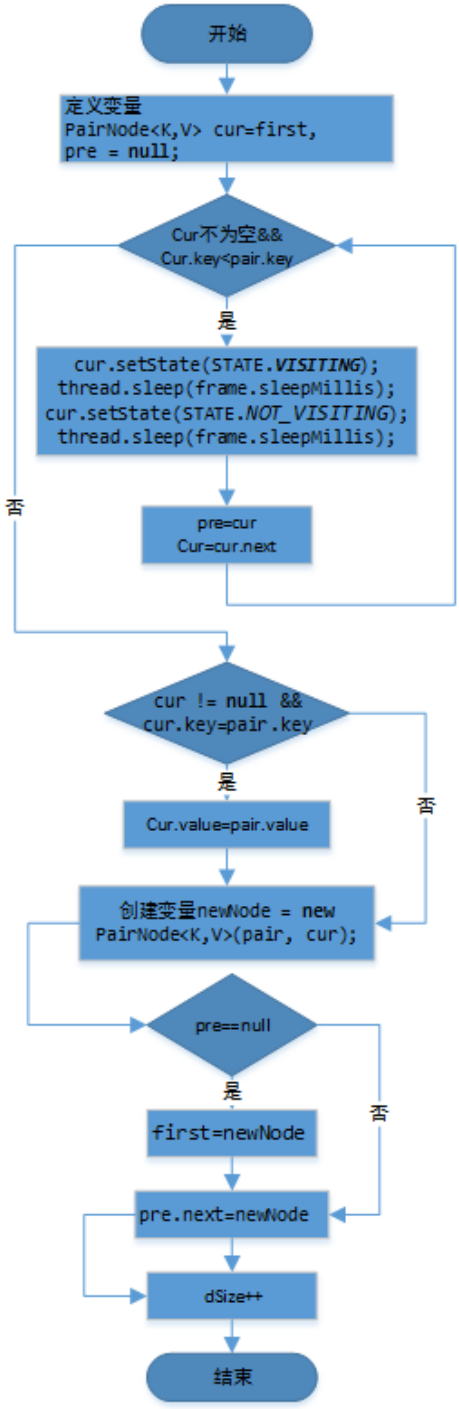
5.2.6HashTable 类型存储结构

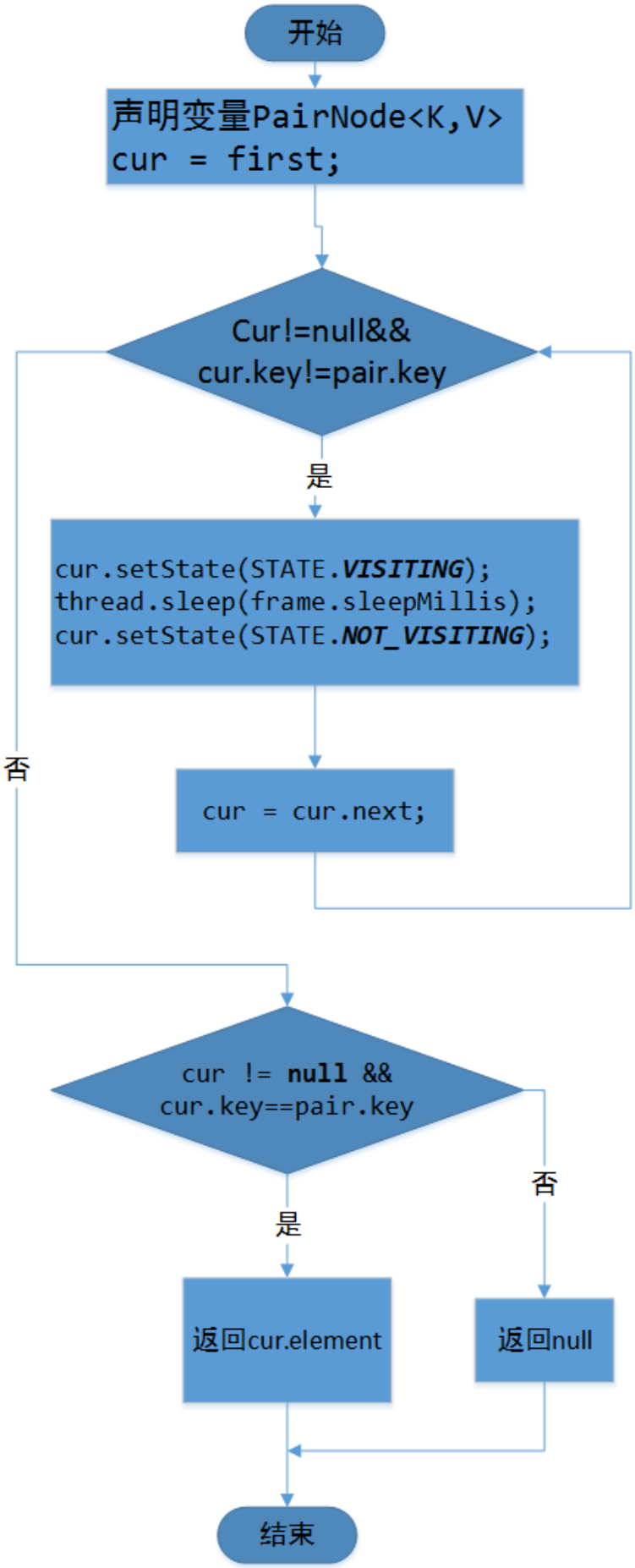
```
public class HashTable<K,V> {

    Pair2<K,V> [] table;
    int dSize;
    int divisor;

    public HashTable(int theDivisor);

    public  void  insert(Pair2<K,V> pair,MainFrame frame,Thread
thread) ;
    public Pair2 search(K key,MainFrame frame,Thread thread) ;
    private int detectKey(K key,MainFrame frame,Thread thread);

    public void draw(Graphics g, int x0, int y0, int nodeSize, int
gap);
    private int hash(Object key) ;
}
```

六、算法设计

6.1 链式哈希表

INSERT(Pair<K,V> pair, MainFrame frame, Thread thread)
SEARCH(Pair<K,V> pair, MainFrame frame, Thread thread)



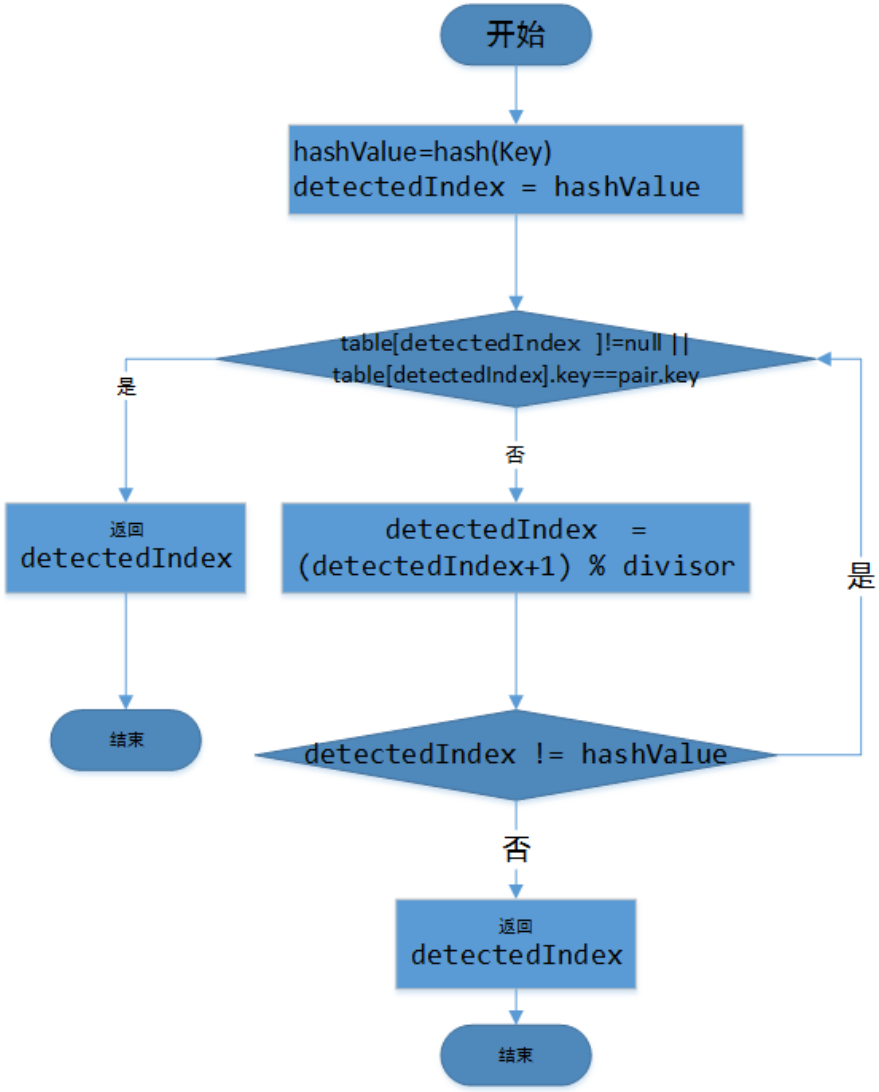

开始
声明变量PairNode<K,V>
cur = first;
Cur!=null&&
cur.key!=pair.key
是
cur.setState(STATE.VISITING);
thread.sleep(frame.sleepMillis);
cur.setState(STATE.NOT_VISITING);
cur = cur.next;
否
cur != null &&
cur.key==pair.key
是
否
返回cur.element
返回null
结束

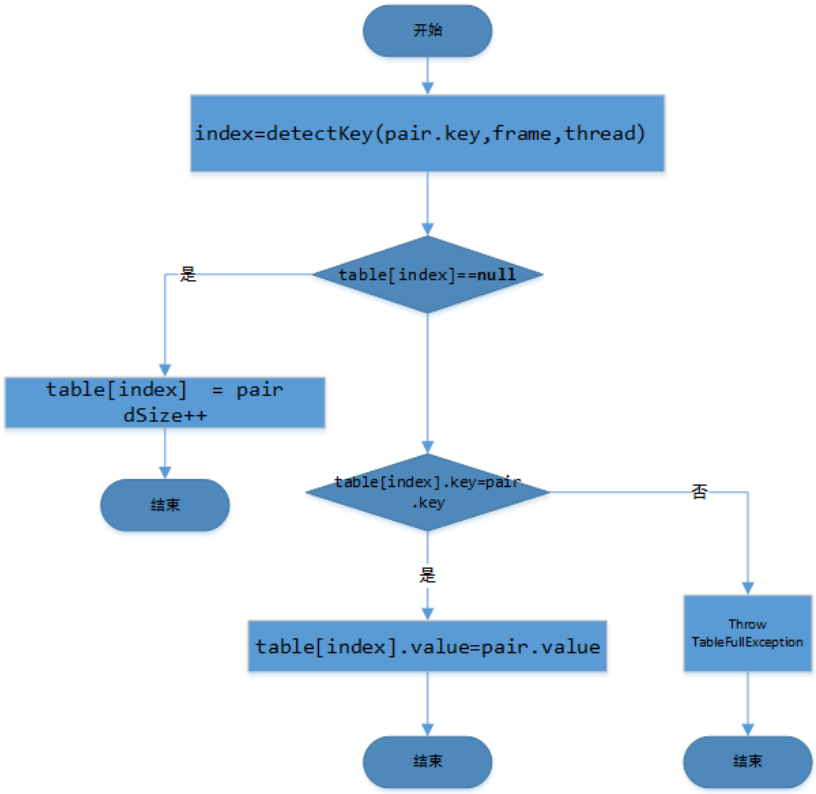
6.2 线性开放寻址

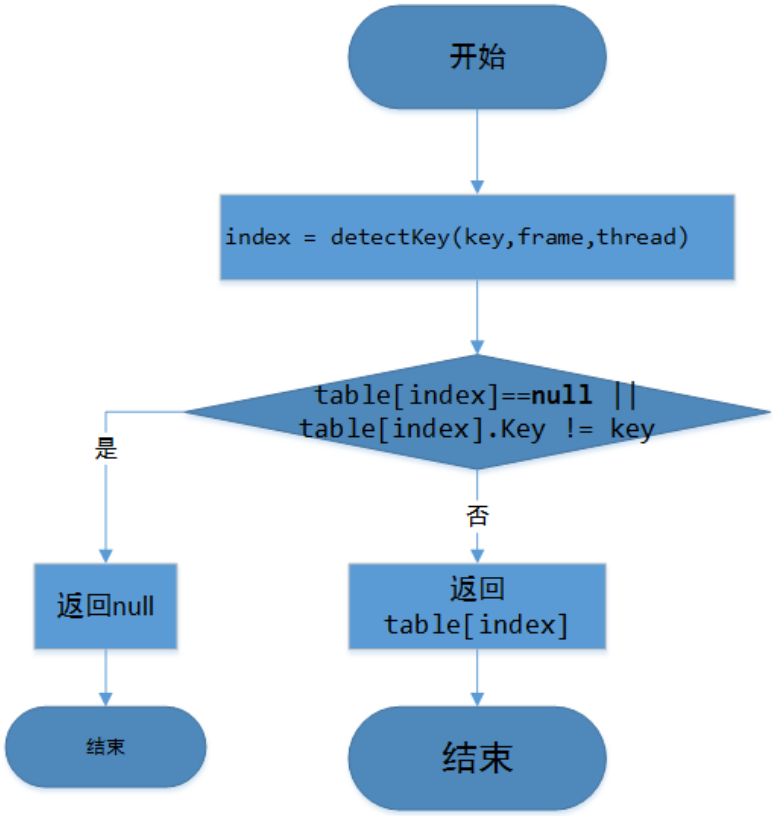
```
DETECT_KEY(K key,MainFrame frame,Thread thread)
```

INSERT(Pair<K,V> pair, MainFrame frame, Thread thread)

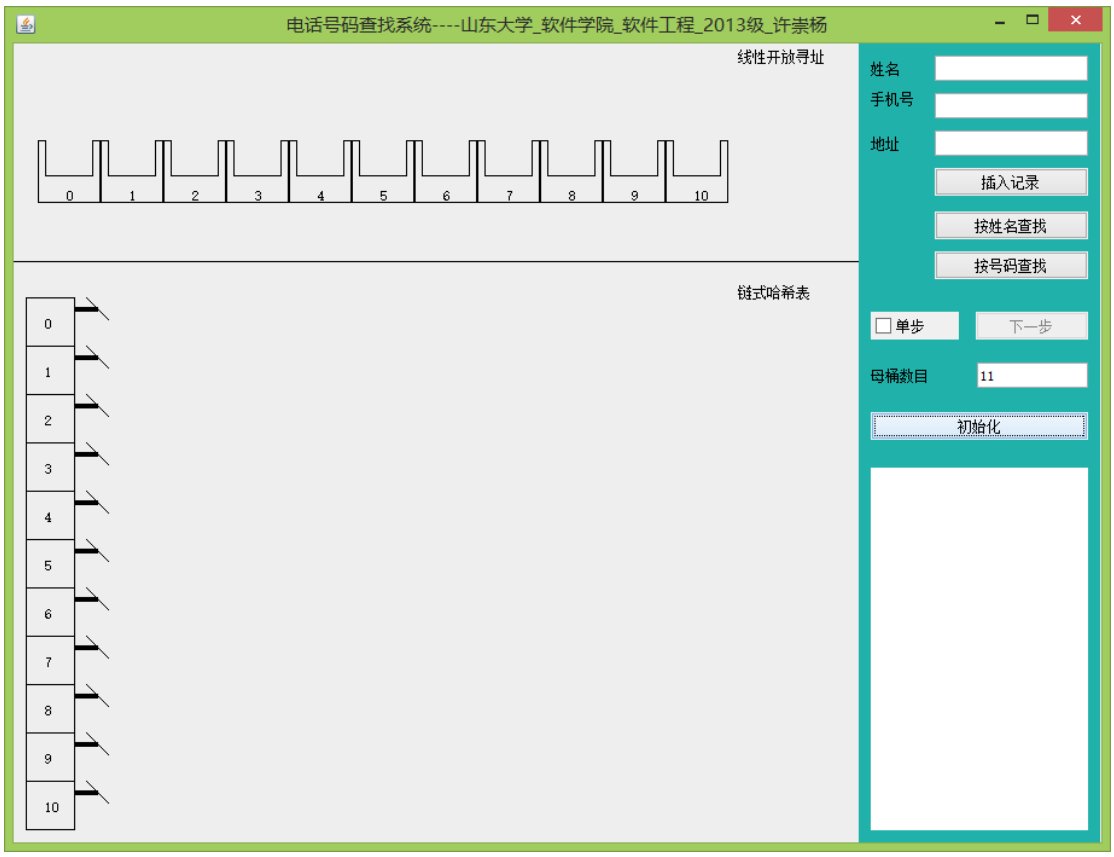
SEARCH(Pair<K,V> pair, MainFrame frame, Thread thread)




开始
index=detectKey(pair.key,frame,thread)
table[index]==null
是
table[index] = pair
dSize++
结束
table[index].key=pair
.key
否
是
table[index].value=pair.value
Throw
TableFullException
结束
结束


开始
index = detectKey(key,frame,thread)
table[index]==null ||
table[index].Key != key
是
否
返回null
返回
table[index]
结束
结束

6.3 界面设计



七、测试和运行

7.1 测试一

测试规模：

母桶个数：11

测试数据：

插入数据：

姓名	手机号码	地址
HC	2	sd
aa	12	山东大学软件园校区
al	30	济南市高新区
许	23	jinan
bk	10	zg
DG	21	china
赵	9	jn
钱	19	hb
孙	4	address

李　　5　　sdu
周　　7　　舜华路
郑　　18　　山东

搜索
按姓名搜索：　许　al　孙　；　许 7
按号码搜索：　19　；　100

7.2 测试二

性能测试

八、总结

这次的开发感受到了写代码之前进行分析设计的作用，写代码之前做了比较充分的需求设计，写代码实现功能的时候比较容易，但是类的设计只是有一些想法，并不够清晰，在功能实现后，觉得类的安排不够合理，又把类重写了，花费了不少时间，如果写代码之前多花一点时间设计清晰，应该可以避免后来的重写
系统的一个特点是系统能够比较清晰的展示哈希表插入和查找的中间过程。系统性能还可以再优化，冲突的解决可以不用有序链表，可以采用效率更高的二叉搜索树，二叉平衡搜索树，如 AVL 树，红黑树等。

附. 参考文献

《Java 高级程序设计语言》
《数据结构，算法与应用——c++语言描述》

附. 程序清单